FPT UNIVERSITY / EMBEDDED LINUX SYSTEMS

LAB-03

Kernel Debugging & Application Performance Analysis

Experimental report | ARM/QEMU + Ubuntu host

REPORT DETAILS	VALUE
Student	Nguyễn Xuân Phú
Student ID (MSSV)	SE203361
Course	Embedded Linux Systems (LOS201)
Date	24 October 2026
Target	QEMU vexpress-a9 / ARMv7 / Linux 5.15.0
Host	Ubuntu, Linux 7.0.0-38-generic (x86-64)

Coverage

PART A	PART B	PART C	PART D
GDB remote debugging	Valgrind & /proc	strace & perf	cyclictest latency

Prepared from the provided source files, terminal logs, profiler outputs, and plot images. Unmeasured claims are distinguished from measured results.

SUMMARY

Executive summary

This report documents four practical investigations in embedded Linux: remote debugging of three C defects on an ARMv7 target, memory-leak diagnosis and correction, syscall/CPU profiling on a native x86-64 host and under QEMU ARM, and scheduling-latency measurements on a non-PREEMPT_RT Linux host.

MEASUREMENT	OBSERVATION
GDB defects	3 observed: out-of-bounds access, dangling pointer, signed integer overflow
Valgrind, initial	16,900 bytes definitely lost in 250 blocks
Valgrind, fixed	0 bytes in use at exit; 0 errors
CPU hotspot	__ieee754_log_fma: 76.84% Self of cpu_core cycle samples in saved perf report
ARM syscall trace	119 calls total; write: 75 calls and 88.72% of measured syscall time
Host latency	Max 80 us (normal), 764 us (stress run 1), 169 us (stress run 2)

Interpretation guardrails. VmRSS can remain unchanged despite allocation leaks; perf percentages refer to sampled CPU events rather than wall-clock shares; strace percentages describe time attributed to syscalls rather than the full application; stress results are nondeterministic; no PREEMPT_RT kernel was tested.

Deliverables and evidence

The submitted material includes C sources and binaries in **apps/**, raw GDB/Valgrind/host strace/perf logs and histogram files in **logs/**, and a bootable QEMU initramfs in **output/**. The ARM strace summary is a condensed manual record; the original full ARM syscall transcript was not included in the archive.

Report structure

1. Environment & method / 2. GDB remote debugging / 3. Memory analysis / 4. Profiling / 5. Real-time latency / 6. Analytical questions / 7. Conclusions and challenges / Appendix: GDB session transcript.

SECTION 01

Environment & methodology

COMPONENT	CONFIGURATION / SOURCE
Native host	Ubuntu x86-64; kernel 7.0.0-38-generic (captured in terminal)
Target	QEMU vexpress-a9 with Cortex-A9, ARMv7, 512 MB RAM, 2 virtual CPUs
Target kernel	Linux 5.15.0; BusyBox-based initramfs
Cross compiler	arm-linux-gnueabihf-gcc (ARM hard-float ABI); debug flags -g
Debugging	gdb-multiarch client; ARM gdbserver listening on forwarded TCP 1234
Memory analysis	Valgrind Memcheck 3.22.0; /proc/self/status on ARM
Profiling	strace; Linux perf; host binary built with -g -O1
Scheduling test	cyclictest 4 SCHED_FIFO threads at priority 80; interval 1000 us
Stress loading	stress-ng: 2 CPU workers, 1 I/O worker, 1 VM worker (128 MB)

The host application was built for x86-64, while ARM applications were cross-compiled for EABI5 hard-float and included in the initramfs. Executables requiring a runtime linker used **/lib/ld-linux-armhf.so.3** and ARM glibc. The ARM target was booted with the original Lab-01 kernel and DTB, plus the Lab-03 initramfs. Binary architectures are verified in the supplied terminal transcript.

Representative host/target workflow

```
arm-linux-gnueabihf-gcc -g -O0 ... -o buggy_app buggy_app.c
gdb-multiarch ./buggy_app
(gdb) target remote 127.0.0.1:1234
sudo perf record -g -o logs/perf.data ./perf_target
sudo cyclictest --threads=4 --priority=80 --histogram=200 ...
```

The report uses only observed run results from the conversation and uploaded material. All latency values are in microseconds (us); VmRSS values are in KiB as reported by /proc. The original command line includes flags omitted in the compact workflow illustration above.

SECTION 02

Part A - GDB remote debugging

2.1 Defect inventory

DEFECT	LOCATION	EVIDENCE	FIX
Array out-of-bounds	buggy_app.c:9-10	i=5, len=5, array=[10,20,30,40,50]	Change i <= len to i < len
Use-after-free	buggy_app.c:22-24	buf remained 0x404198 after free; rendered bytes changed	Return live allocation; free once in main
Signed overflow	buggy_app.c:29	int result=1,410,065,408; 64-bit product=10,000,000,000	Promote operand before multiplication

2.2 Bug 1 - Out-of-bounds access

The function **process_data** iterated over a five-element array using **i <= len**, so the iteration with **i = len = 5** attempts to read **data[5]**, outside indices 0..4. GDB stopped at line 10 before this read. The valid elements sum to 150. The behavior is undefined even if the program happens to continue.

```
Breakpoint 3, process_data (data=0x7efffd48, len=5) at buggy_app.c:10
$1 = 5                    # i
$2 = 5                    # len
$3 = 150                  # sum
$4 = {10, 20, 30, 40, 50} # valid array contents
#0 process_data (...) at buggy_app.c:10
#1 main () at buggy_app.c:43
```

2.3 Bug 2 - Use-after-free

create_message allocated and copied a string, then called **free(buf)** and returned the same pointer at line 24. The GDB session shows the pointer address unchanged while the bytes GDB rendered at that address changed after free. Accessing it after free is not permitted; altered bytes are an observation, not a defined outcome.

```
Before free: $5 = 0x404198 "Hello Embedded Linux"
After free:  $6 = 0x404198 "\004\004"
#0 create_message (...) at buggy_app.c:24
#1 main () at buggy_app.c:48
```

2.4 Bug 3 - Signed integer overflow

The input parameters **a = b = 100,000** produce an exact product of 10,000,000,000, beyond the largest signed 32-bit integer (2,147,483,647). The observed 32-bit result is 1,410,065,408. Signed overflow in C is undefined; that numerical result must not be treated as portable wraparound behavior.

```
Breakpoint 6, calculate (a=100000, b=100000) at buggy_app.c:29
$7 = 1410065408                  # result (int)
$8 = 10000000000                 # (long long)a * (long long)b
#0 calculate (...) at buggy_app.c:31
#1 main () at buggy_app.c:55
```

2.5 Corrections and verification

The supplied **buggy_app_fixed.c** uses **i < len**, returns the live heap string and frees it in **main()**, and calculates the product with **long long result = (long long)a * b**. The fixed ARM executable was tested in QEMU; the corrected total is 150 and the product is 10,000,000,000. The later GDB session ended normally. The complete saved GDB output, including frame and register information, is reproduced in Appendix A.

SECTION 03

Part B - Memory analysis

3.1 Memcheck findings before fixing

LOSS SITE	DIRECT LOSS	SOURCE TRACE
scores (100 allocations)	2,000 bytes / 100 blocks	create_record(); leaky_app.c:25
name (100 allocations)	2,100 bytes / 100 blocks	create_record(); leaky_app.c:22
message buffers (50 allocations)	12,800 bytes / 50 blocks	process_messages(); leaky_app.c:50
Total	**16,900 bytes / 250 blocks**	**Definitely lost**

In **delete_record()**, the original code freed the Record structure but did not free its **name** and **scores** allocations. In **process_messages()**, 50 buffers of 256 bytes were never released. Memcheck reported **352 allocations, 102 frees** in the original and three loss contexts. No indirectly lost blocks were reported in this run.

```
definitely lost: 16,900 bytes in 250 blocks
indirectly lost:      0 bytes in 0 blocks
possibly lost:        0 bytes in 0 blocks
still reachable:      0 bytes in 0 blocks
ERROR SUMMARY: 3 errors from 3 contexts
```

3.2 Corrected application

The corrected implementation frees **r->name**, **r->scores**, then **r**, and frees each temporary message buffer inside the loop. It also checks allocation results. The fixed Memcheck log verifies that all blocks were freed.

```
in use at exit: 0 bytes in 0 blocks
total heap usage: 352 allocs, 352 frees, 21,124 bytes allocated
All heap blocks were freed -- no leaks are possible
ERROR SUMMARY: 0 errors from 0 contexts
```

3.3 Target memory readings from /proc/self/status

FIELD	BEFORE	AFTER	DELTA
VmSize	1,548 KiB	1,548 KiB	0 KiB
VmRSS	156 KiB	156 KiB	0 KiB
VmData	192 KiB	192 KiB	0 KiB

The ARM monitor process (PID 57) read **/proc/self/status** before and after the allocation/leak workload and exited with code 0. No difference was reported at the granularity of these kernel memory fields. That does **not** disprove a 16,900-byte leak: VmRSS accounts for resident pages, heap allocator reuse, page commitment, and preexisting mappings, whereas Memcheck tracks allocation reachability.

> **Methodological limit.** This is a before/after measurement within one run of the leaking program, not a comparison between the leaking and corrected ARM programs. The leak classification comes from host Memcheck; /proc provides complementary memory-footprint context.

SECTION 04

Part C - strace & perf profiling

4.1 Host syscall trace (x86-64)

SYSTEM CALL	TIME (%)	CALLS	TIME (s)
execve	53.02	1	0.000413
mmap	19.90	14	0.000155
mprotect	5.52	4	0.000043
openat	3.34	4	0.000026
fstat	2.82	5	0.000022

The host trace counted **124 syscalls**, with **0.000779 s** of cumulative measured syscall time and one reported error. Although **write** was called 75 times, it accounted for only 1.80% of the syscall time in this measurement. Buffered **fprintf()** converts 10,000 logical lines into fewer kernel writes. The percentages above do not represent fractions of the total application runtime.

4.2 ARM/QEMU syscall trace

SYSTEM CALL	TIME (%)	CALLS	TIME (s)
write	88.72	75	0.004884
munmap	9.99	2	0.000550
close	1.29	3	0.000071
All syscalls	**100.00**	**119**	**0.005505**

On the emulated ARM Linux 5.15 target, **strace -c** returned successfully, reporting **119 syscalls** and **7 syscall error returns**. The package contains a condensed summary with the three nonzero-time syscall categories, not the full raw trace. ARM/QEMU execution took about **0.81 s wall time** versus **0.03333 s** for the instrumented host **perf stat** run. Different instruction sets, emulation, I/O environments, and instrumentation prevent a controlled CPU hardware-speed comparison.

4.3 perf report: measured hotspots

The saved **perf_report.txt** is the later run: **92 cpu_core/cycles/P samples** and **2 cpu_atom/cycles/P samples**. The table uses the larger **cpu_core** event section. “Self” is the direct sampled event share for the symbol, distinct from “Children”, which includes descendants. Tied lower entries are shown as recorded.

SYMBOL	SHARED OBJECT	SELF (%)
__ieee754_log_fma	libm.so.6	76.84
heavy_compute	perf_target	8.18
log@@GLIBC_2.29	libm.so.6	6.48

SYMBOL	SHARED OBJECT	SELF (%)
__log_finite@GLIBC_2.15@plt	libm.so.6	1.19
__vfprintf_internal	libc.so.6	1.19

The dominant CPU hotspot is the math-library logarithm path. **heavy_compute()** evaluates **sqrt(arr[i]) * log(arr[i]+1)** over 100,000 values for 50 passes. Distinct symbols on a call path should not be interpreted as independent wall-clock shares. With 92 core samples, rankings beyond the dominant hotspot have sampling uncertainty.

4.4 perf stat and optimization priority

COUNTER / METRIC	RECORDED VALUE
Elapsed wall time	0.033330625 s
Task clock	32.18 ms
cpu_core cycles	94,944,083
cpu_core instructions	465,409,266
cpu_core instructions/cycle	4.9 (reported rounded)
Context switches / migrations	1 / 1
Page faults	371
Sample coverage	cpu_core counters multiplexed at ~70.15%; cpu_atom at ~14.3%

The numerical kernel of **heavy_compute()** is the first target for optimization. Candidate changes include caching/precomputing invariant mathematical terms across the 50 repeated passes, suitable vector math / NEON on ARM where available, and careful compiler optimization and loop restructuring. Measure each candidate on the intended target; the present logs do not contain a before/after optimized perf benchmark. The unused **linear_search** result may allow the optimizing compiler to eliminate work, so its advertised 1000 iterations are not proof of CPU cost.

SECTION 05

Part D - Real-time scheduling latency

5.1 Measured latency by load condition

MEASUREMENT	NORMAL	STRESS #1	STRESS #2
Min latency (us)	2	2	2
Avg latency (us)	~3	~3	~3 to 4
Max latency (us)	80	764	169
In-range histogram entries	40,000	39,996	40,000
Histogram overflows	0	4	0
Histogram range (us)	0-199	0-199	0-1999

All runs used four priority-80 FIFO threads and a 1,000 us base interval. Under stress-ng load (2 CPU, 1 I/O, 1 VM workers), the first trial recorded a maximum of **764 us** across threads; its 200-us histogram excluded four out-of-range observations. A second stress trial used a 2,000-us histogram and captured all 40,000 entries; its maximum was **169 us**. Repeated-run variation is expected under non-deterministic scheduling and background activity.

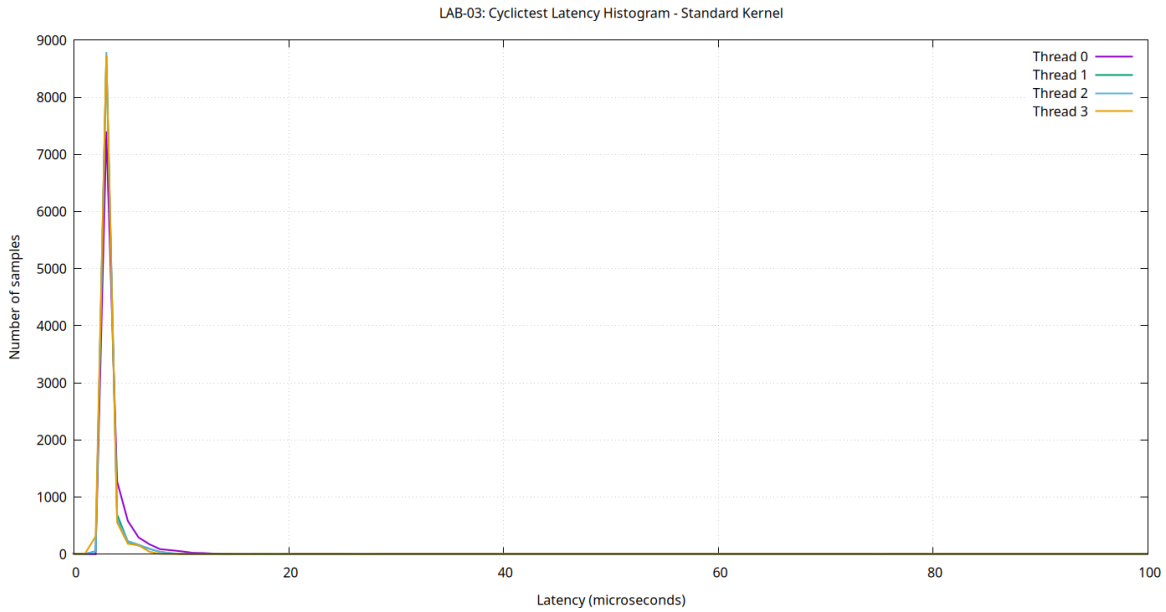


Figure 1. Original normal-load latency histogram; four measured threads, 0-100 us displayed.

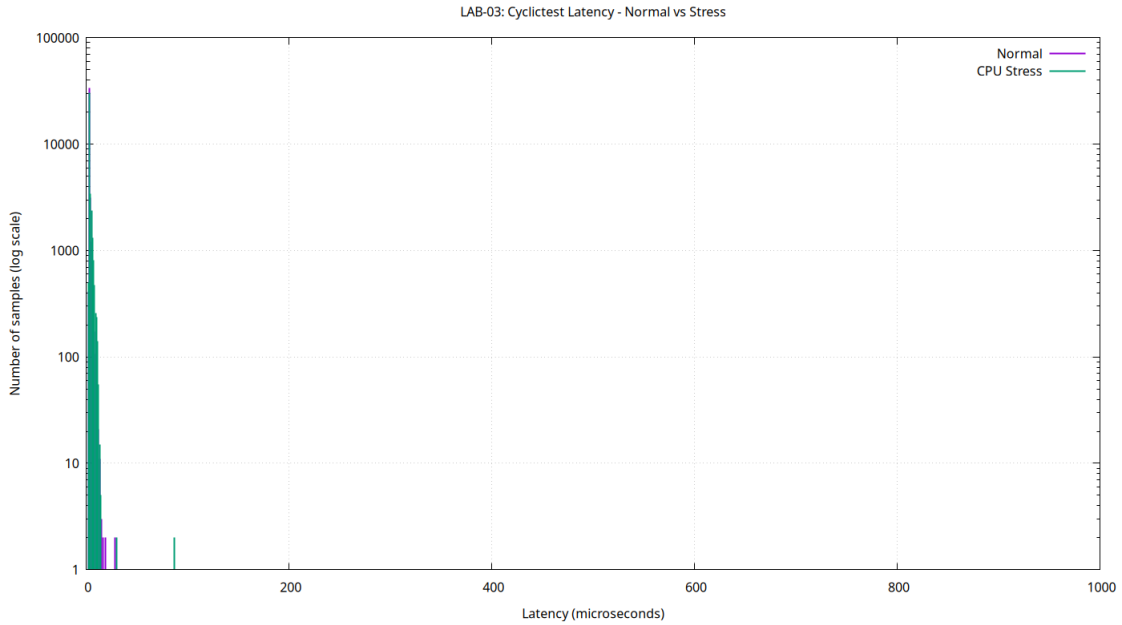


Figure 2. Supplied log-scale histogram comparing normal load with stress run #2 (not stress run #1).

5.2 Tail-latency interpretation

The maximum rose from **80 us** in the normal trial to **764 us** in stress trial #1 (9.55x). This can matter for a task with a strict 500-us wakeup-latency limit even though rounded average latency stayed near 3 us. In the 40,000 normal-trial histogram observations, **99.995%** were below 50 us. In the full-range stress trial #2, **99.955%** were below 50 us. These percentages summarize binned samples, not statistical guarantees of future real-time behavior.

5.3 Kernel configuration and PREEMPT_RT theory

```
Kernel: 7.0.0-38-generic
CONFIG_PREEMPT_BUILD=y
CONFIG_PREEMPT_LAZY=y
CONFIG_PREEMPT_DYNAMIC=y
# CONFIG_PREEMPT_RT is not set
CONFIG_HZ=1000
```

This host is not running a PREEMPT_RT kernel. The configured dynamic/lazy preemption features are not equivalent to **CONFIG_PREEMPT_RT=y**. PREEMPT_RT generally reduces long non-preemptible sections through threaded interrupt handling and real-time-aware locking, which may reduce worst-case latency under relevant workloads. Neither a PREEMPT_RT build nor comparative cyclictest measurements on one were supplied; no measured RT latency is claimed.

> **Robotics example.** For a control loop needing wakeup/response below 500 us, trial #1 already showed a 764-us spike on this generic host. A properly engineered PREEMPT_RT or dedicated RTOS configuration is a candidate, but the end-to-end deadline must be verified with target hardware, interrupt priorities, load testing, and worst-case analysis.

SECTION 06

Answers to analytical questions

Q1. What happens after “next” in remote GDB?

The **gdb-multiarch** client calculates what “step over” means at the current source location. Depending on the instruction mapping, it may install a temporary breakpoint at a later instruction or arrange stepping, then send remote commands over a TCP connection to **gdbserver** (host-forwarded port 1234). GDB and gdbserver communicate using the **GDB Remote Serial Protocol (RSP)**, with packets for registers, memory, stepping, breakpoint handling, and stop notifications. gdbserver resumes the inferior on ARM Linux; the process runs under QEMU until a trap/signal/temporary breakpoint, then gdbserver reports the stop to the host. GDB fetches registers and memory as needed, resolves ARM debug symbols, and displays the new source line. Multiarch support is important because the host itself is x86-64 while the target ELF, registers, and instruction set are ARMv7. The recorded session demonstrates this chain at source breakpoints and the remote link **127.0.0.1:1234**.

Q2. “Definitely lost” versus “indirectly lost”

Definitely lost means Valgrind cannot find a usable pointer to an allocated block at process exit. **Indirectly lost** means a block is only reachable through another lost allocation; e.g., if a parent structure itself leaks and points to child blocks. Here **delete_record()** frees the parent **Record** first, without freeing **name** and **scores**. The saved trace classifies the child buffers as **definitely lost**: name 2,100 bytes, scores 2,000 bytes, plus 12,800 bytes from **process_messages()**. The actual indirect-loss total is zero. Memcheck can also identify invalid reads/writes, use of uninitialized values, and invalid/double frees, in addition to leaks.

Q3. How could heavy_compute() be optimized?

Option 1 - remove repeated work: in this supplied benchmark **darr** does not change between the 50 passes. Compute the per-element expression or the aggregate once and reuse it, if application semantics allow; this removes most repeated transcendental evaluations. An idealized upper bound approaches a many-fold reduction of the compute-loop work (up to roughly the 50 passes), but not a 50x end-to-end application speedup. **Option 2 - vectorization:** on ARMv7, use suitable NEON/SIMD techniques, alignment, and vectorized math libraries for logarithm/square root where accurate implementations exist. Potential gains are workload- and implementation-dependent; even a hypothetical 1.2-3x improvement of the vectorized portion requires a target benchmark. **Option 3 - compiler/loop tuning:** benchmark -O2/-O3, function inlining, loop unrolling, and locality; preserve required numerical precision. The hotspot **__ieee754_log_fma (76.84% Self)** makes these math paths the first place to test, but no speedup was measured in this lab.

Q4. Why do standard kernels show latency spikes?

Interrupts and softirqs: a time-consuming interrupt handler or bottom half can postpone a higher-priority user task. PREEMPT_RT threads most interrupt handlers so they can be scheduled and prioritized, although some low-level IRQ sections remain non-threaded. **Scheduler/preemption:** long kernel critical sections and preemption-disabled paths delay an awakened real-time thread; RT preemption narrows these windows. **Memory and locking:** page faults, allocation/reclaim, and contended locks can introduce unpredictable delays; memory locking reduces faults, and RT-aware lock primitives enable priority inheritance in many sleeping-lock cases. The measured 764-us peak is consistent with a latency spike but the measurement alone does **not** attribute its cause to any single mechanism.

Q5. Select the right diagnosis tool

PROBLEM	BEST FIT	REASON
(a) Random crash hours later	GDB (+ logs/core dumps)	Reconstruct call stack, arguments, registers and faulting instruction; use remote GDB on ARM.
(b) Progressively slower run	Valgrind (+ /proc)	Identify unreleased allocations; trend process size as complementary evidence.
(c) Occasional deadline miss	cyclictest	Measure distribution and worst observed scheduling latency under load; inspect RT kernel suitability.
(d) One function slows system	perf (strace if syscall-heavy)	Sample CPU cycles and call stacks; use strace when time is dominated by syscall activity.

The observed evidence follows this division: GDB exposed three ARM C defects; Memcheck identified and then eliminated 16,900 leaked bytes; perf found an expensive math-library path; cyclictest exposed a 764-us scheduling-latency excursion under stress.

SECTION 07

Conclusion, challenges & limitations

7.1 Main findings

- Remote ARM debugging recovered the precise line and runtime values for all three demonstration defects; source-level fixes were prepared and tested.
- Memcheck documented a real lifetime-management bug with 16,900 bytes definitely lost, reduced to zero in the corrected host binary.

- Native perf identified logarithm computations as the principal sampled CPU-cycle hotspot; ARM strace successfully collected syscall timing under QEMU.
- Scheduling latency showed a low average but a substantial stress-induced tail (up to 764 us). The tested host did not use PREEMPT_RT.

7.2 Practical difficulties and resolutions

DIFFICULTY	RESOLUTION / LEARNING
ARM tools missing from BusyBox rootfs	Copied ARM gdbserver and strace plus required runtime libraries; repackaged initramfs.
GDB remote-session evidence not saved initially	Enabled GDB logging, reran breakpoints, and saved full output to gdb_session.txt.
perf_target file I/O initially denied	Removed the conflicting /tmp output; reran perf stat and verified 10,000 output lines.
Histogram overflow during stress	Expanded buckets from 200 to 2,000 us, producing a zero-overflow second stress trial.

7.3 Limitations & submission checklist

- The saved ARM strace text is a hand-recorded condensed summary, not a full raw syscall trace.
- perf sampled few events (especially only two cpu_atom samples), and perf stat hardware counters were multiplexed; benchmark repetitions would improve confidence.
- The two stress trials differ in duration and timing and do not establish a deterministic worst-case guarantee.
- No PREEMPT_RT kernel was installed or measured, so RT comparisons are theoretical.
- Student information is included; add class/group details if required by the instructor.

7.4 Evidence index

apps/ includes original and fixed C files and ARM/x86-64 executables; **logs/gdb_session.txt** contains remote debug output; **logs/valgrind_report.txt** and **valgrind_fixed.txt** contain heap checks; **logs/perf_report.txt**, **perf_stat.txt**, **strace_summary.txt**, and **strace_arm_summary.txt** contain profiling evidence; **logs/hist_standard.txt**, **hist_stress.txt**, and **hist_stress_full.txt** contain histogram readings; supplied PNG files contain visualizations; **output/initramfs_lab3.cpio.gz** is the QEMU boot image.

References. LAB-03 practical handout (user-provided course instructions); GDB Remote Protocol documentation; Valgrind Memcheck manual; Linux perf tool documentation; cyclictest/rt-tests manual; Linux PREEMPT_RT documentation. These are contextual reference names, not substituted for observed local logs.

APPENDIX A

GDB remote-session transcript

The following reproduces all substantive lines in the submitted **logs/gdb_session.txt**, preserving GDB output. Terminal carriage-return progress indicators were removed for readability; the original raw file remains in the submitted materials. GDB logging captured results but did not echo every typed command.

```
The target architecture is set to "arm".
Remote debugging using 127.0.0.1:1234
Reading symbols from /home/phu/embedded_lab3/rootfs/initramfs/lib/ld-linux-armhf.so.3...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x76fe0000

0x76ff0598 in _start () from /home/phu/embedded_lab3/rootfs/initramfs/lib/ld-linux-armhf.so.3
Breakpoint 1 at 0x40072e: file buggy_app.c, line 36.
Continuing.
Downloading separate debug info for /home/phu/embedded_lab3/rootfs/initramfs/lib/libc.so.6

Breakpoint 1, main () at buggy_app.c:36
36    int main(void) {
on: Logging is enabled.
Already logging to /home/phu/embedded_lab3/logs/gdb_session.txt.
Breakpoint 2 at 0x400682: file buggy_app.c, line 7.
Continuing.

Breakpoint 2, process_data (data=0x7efffd48, len=5) at buggy_app.c:7
7        int sum = 0;
data = 0x7efffd48
len = 5
1    #include <stdio.h>
2    #include <stdlib.h>
3    #include <string.h>
4
5    /* Bug 1: Array out-of-bounds */
6    void process_data(int *data, int len) {
7        int sum = 0;
8
9        for (int i = 0; i <= len; i++) {
10           sum += data[i];
11       }
12
13       printf("Sum = %d\n", sum);
14   }
15
16   /* Bug 2: Use-after-free */
17   char *create_message(const char *text) {
18       char *buf = malloc(strlen(text) + 1);
19
20       strcpy(buf, text);
Breakpoint 3 at 0x40068c: file buggy_app.c, line 10.
Continuing.

Breakpoint 3, process_data (data=0x7efffd48, len=5) at buggy_app.c:10
10           sum += data[i];
Undefined command: "10".  Try "help".
$1 = 5
$2 = 5
$3 = 150
$4 = {10, 20, 30, 40, 50}
#0  process_data (data=0x7efffd48, len=5) at buggy_app.c:10
#1  0x00400770 in main () at buggy_app.c:43
i = 5
sum = 150
r0             0x7efffd48          2130705736
r1             0x5                 5
r2             0x5                 5
r3             0x5                 5
r4             0x7efffd58          2130705752
```

```
r5             0x40097c            4196732
r6             0x401eac            4202156
r7             0x7efffd28          2130705704
r8             0x400729            4196137
r9             0x76ffecd8          1996483800
r10            0x0                 0
r11            0x401eac            4202156
r12            0x4                 4
sp             0x7efffd28          0x7efffd28
lr             0x400771            4196209
pc             0x40068c            0x40068c <process_data+20>
cpsr           0x600f0030          1611595824
fpscr          0x0                 0
11        }
14    }
Breakpoint 4 at 0x4006cc: file buggy_app.c, line 18.
Continuing.

Breakpoint 4, create_message (text=0x400920 "Hello Embedded Linux") at buggy_app.c:18
18        char *buf = malloc(strlen(text) + 1);
16    /* Bug 2: Use-after-free */
17    char *create_message(const char *text) {
18        char *buf = malloc(strlen(text) + 1);
19
20        strcpy(buf, text);
21
22        free(buf);
23
24        return buf;
25    }
26
27    /* Bug 3: Integer overflow */
28    int calculate(int a, int b) {
29        int result = a * b;
30
31        printf("calculate(%d, %d) = %d\n", a, b, result);
32
33        return result;
34    }
35
36    int main(void) {
37        printf("=== Buggy App v1.0 - LAB-03 ===\n");
38
text = 0x400920 "Hello Embedded Linux"
#0  create_message (text=0x400920 "Hello Embedded Linux") at buggy_app.c:18
#1  0x00400784 in main () at buggy_app.c:48
Temporary breakpoint 5 at 0x4006e8: file buggy_app.c, line 22.
Continuing.

Temporary breakpoint 5, create_message (text=0x400920 "Hello Embedded Linux") at buggy_app.c:22
22        free(buf);
$5 = 0x404198 "Hello Embedded Linux"
0x404198:    "Hello Embedded Linux"
24        return buf;
$6 = 0x404198 "\004\004"
buf = 0x404198 "\004\004"
#0  create_message (text=0x400920 "Hello Embedded Linux") at buggy_app.c:24
#1  0x00400784 in main () at buggy_app.c:48
Breakpoint 6 at 0x400702: file buggy_app.c, line 29.
Continuing.

Breakpoint 6, calculate (a=100000, b=100000) at buggy_app.c:29
29        int result = a * b;
Undefined command: "Breakpoint".  Try "help".
a = 100000
b = 100000
27    /* Bug 3: Integer overflow */
28    int calculate(int a, int b) {
29        int result = a * b;
30
31        printf("calculate(%d, %d) = %d\n", a, b, result);
32
33        return result;
34    }
31        printf("calculate(%d, %d) = %d\n", a, b, result);
$7 = 1410065408
```

```
$8 = 10000000000
#0  calculate (a=100000, b=100000) at buggy_app.c:31
#1  0x004007b0 in main () at buggy_app.c:55
result = 1410065408
r0             0x186a0             100000
r1             0x186a0             100000
r2             0x186a0             100000
r3             0x540be400          1410065408
r4             0x7efffd58          2130705752
r5             0x40097c            4196732
r6             0x401eac            4202156
r7             0x7efffd28          2130705704
r8             0x400729            4196137
r9             0x76ffecd8          1996483800
r10            0x0                 0
r11            0x401eac            4202156
r12            0x4                 4
sp             0x7efffd28          0x7efffd28
lr             0x4007b1            4196273
pc             0x40070c            0x40070c <calculate+20>
cpsr           0x200f0030          537854000
fpscr          0x0                 0
Continuing.
[Inferior 1 (process 60) exited normally]
```

End of GDB transcript. Other original logs and the complete perf.data files remain in the attached material archive.